VacunaApp: base de datos, costos y migración a Postgres propio

Documento de referencia. Actualizado: 15 de octubre de 2026. Los precios cambian: confírmalos en las páginas oficiales enlazadas en cada sección antes de decidir.

---

1. Dónde está el proyecto hoy

- **Aplicación:** Next.js en Vercel, con recordatorios por Email (Resend) y WhatsApp (Twilio).
- **WhatsApp:** funciona solo con Twilio. Se quitó la integración directa con la API de Meta (`docs/WHATSAPP_META.md` y la ruta `/api/whatsapp/meta` ya no existen).
- **Estado real de entrega:** Twilio avisa a `/api/whatsapp/estado` si cada WhatsApp se entregó o falló. Un recordatorio que Twilio aceptó pero no entregó pasa a "Fallido" con el motivo (antes quedaba "Enviado" aunque no llegara).
- **Suscripciones de los centros:** cada centro tiene plan y fecha de vencimiento. Paga con links fijos de Mercado Pago (mensuales y anuales) y el superadministrador confirma cada pago en `/plataforma`. Al vencer hay 1 día de gracia y luego se bloquea el panel. Hay un modo automático (Checkout Pro + webhook) que solo se activa con `MERCADOPAGO_ACCESS_TOKEN`.
- **Base de datos:** Supabase (Postgres + login + almacenamiento de logos + tiempo real).
- **Esquema en un solo archivo:** `supabase/vacunaapp_esquema_completo.sql`.

Qué hace el archivo `vacunaapp_esquema_completo.sql`

- Reemplaza a los `.sql` anteriores (schema, roles, confirmaciones, establecimiento, autorizacion, campanas, carne, dosis_externas, multicentro, inventario, suscripciones y estado_whatsapp).
- **Base nueva:** crea el esquema final completo.
- **Base ya en uso:** la actualiza sin perder datos. Se puede ejecutar varias veces sin riesgo.
- **PostgreSQL puro:** incluye una sección de compatibilidad que crea los roles `anon`, `authenticated` y `service_role`, y el esquema `auth` (`auth.users` y `auth.uid()`) si no existen. Omite Storage y Realtime, que son de Supabase.
- **No incluye** los datos de demostración (`supabase/seed.sql`).
- Requiere PostgreSQL 15 o superior.

Cómo ejecutarlo:

- **Supabase:** Editor SQL > pegar todo el archivo > Run. Haz un respaldo antes.
- **PostgreSQL propio:** `psql -d vacunaapp -v ON_ERROR_STOP=1 -f supabase/vacunaapp_esquema_completo.sql`

Se probó en una base de pruebas local (PGlite): la estructura resultante es igual a la de los archivos anteriores, los datos se conservan al migrar, y funcionan el aislamiento entre centros, el inventario por lote

y los permisos por rol. No se probó contra tu proyecto real de Supabase ni contra PostgreSQL de servidor: haz siempre un respaldo primero.

---

2. ¿Funcionaría igual si todo pasa a Postgres?

La base de datos sí. La aplicación no, sin cambios en el código.

Pieza	En Postgres puro
Tablas, triggers, reglas de inventario	Funcionan igual (las cubre el archivo SQL)
Aislamiento entre centros y permisos por rol (RLS)	Funcionan, pero la conexión debe declarar el usuario con `set_config('request.jwt.claim.sub', ...)`
API de datos de Supabase (PostgREST)	**No existe.** La app usa `supabase-js` (`.from("pacientes")`) en unos 56 archivos
Login de Supabase (`auth`)	**No existe.** Hay que implementar un login propio o usar una librería
Storage (logos)	**No existe.** Hay que guardar los logos en disco o en otro servicio
Realtime (monitor y agenda en vivo)	**No existe.** Se reemplaza por consultas periódicas o `LISTEN/NOTIFY`

Es decir: mover los datos es fácil; lo que cuesta trabajo es cambiar la capa de código que habla con Supabase.

---

3. Costos

Supabase (datos en su nube)

Plan	Precio	Incluye	Para VacunaApp
Free	$0	500 MB de base, 1 GB de archivos, 5 GB de transferencia	**No sirve para producción:** se pausa tras 1 semana sin uso y no tiene respaldos
Pro	desde $25/mes	8 GB de base, 100 GB de archivos, 250 GB de transferencia, respaldos diarios (7 días), sin pausas	Mínimo razonable para datos de pacientes

Con Pro, el costo sube solo si superas esos límites (por ejemplo, $0.125 por cada GB extra de base).

¿Hace falta hosting aparte? Sí, para la aplicación

Supabase guarda datos, login y logos. El código de VacunaApp (Next.js) debe correr en un servidor siempre encendido: los recordatorios automáticos (cron) y el webhook de Twilio lo necesitan.

Opción	Costo mensual aproximado
Supabase Pro + Vercel Pro	**≈$45** ($25 + $20)
Supabase Pro + Vercel Hobby	≈$25. Hobby es solo para uso no comercial: no aplica a un centro de vacunación
Supabase Pro + VPS propio	≈$25 + VPS (estimado $5–12; este valor no sale de las páginas oficiales)
Todo en VPS propio (Postgres propio)	Solo el VPS (estimado $6–15) + dominio + respaldos

Costos aparte, sin importar el hosting

Los valores en pesos usan un dólar de referencia de **$4.000 COP**; ajústalos a la tasa del día.

WhatsApp (Twilio + Meta), por centro

Concepto	Costo
Tarifa de Twilio por cada mensaje enviado **o recibido**	$0,005 USD
Tarifa de Meta por plantilla de utilidad en Colombia (el recordatorio lo es)	≈ $0,0034 USD
Plantillas de utilidad dentro de las 24 h después de que el paciente escribe	Gratis para Meta
Respuestas del bot (mensajes de servicio, desde octubre de 2026)	Primeras 1.000 al mes por número gratis para Meta
Registrar el número del centro como remitente en Twilio	Sin costo de Twilio
Número comprado en Twilio (si el centro no usa el suyo)	≈$1–2 USD/mes (no verificado para números de Colombia)

Ejemplo, centro con 300 recordatorios al mes que casi todos responden: 300 recordatorios × ($0,005 + $0,0034) + 300 respuestas × $0,005 + 300 respuestas del bot × $0,005 ≈ **$5,5 USD (≈$22.000 COP) al mes**.

Requisitos para que los mensajes lleguen a pacientes reales: un número aprobado en Twilio con cuenta de Meta Business verificada, y las plantillas aprobadas por Meta. El sandbox actual solo entrega a quien escribió al número en las últimas 24 horas (error 63016) o se unió con `join …` (error 63015).

Fuentes: Twilio – WhatsApp Pricing · Twilio – Cambios de precios de octubre de 2026 · Meta – Pricing on the WhatsApp Business Platform

Email (Resend)

Plan	Precio	Incluye
Free	$0	3.000 correos al mes y máximo 100 al día
Pro	$20 USD/mes (≈$80.000 COP)	50.000 correos al mes, sin límite diario

- Con el plan gratis y sin dominio verificado, Resend **solo entrega a la cuenta dueña** (hoy, el correo personal del superadministrador). Para enviar recordatorios a pacientes hace falta **verificar un**

dominio.

- Con varios centros, el límite de 100 correos al día del plan gratis se alcanza rápido: el cron envía todo una vez al día.

Fuente: Resend – Pricing

Dominio

Un dominio propio (por ejemplo `vacunaapp.co`) cuesta alrededor de **$40.000–60.000 COP al año** (estimado; no sale de una página oficial). Sirve para verificar el correo en Resend y para la dirección de la plataforma.

Cobro de suscripciones (Mercado Pago, link de pago)

Sin costo fijo. Comisión por pago, según cuándo quieras el dinero disponible:

Disponible	Comisión
Al instante	3,29 % + IVA
En 7 días	2,99 % + IVA
En 14 días	2,79 % + IVA

Pueden aplicar además retenciones de impuestos según tu situación tributaria.

Plan	Precio	Comisión al instante (3,29 % + IVA)	Comisión a 14 días (2,79 % + IVA)
Básico mensual	$280.000	≈$10.960	≈$9.300
Pro mensual	$480.000	≈$18.790	≈$15.940
Enterprise mensual	$980.000	≈$38.370	≈$32.540
Básico anual	$2.800.000	≈$109.620	≈$92.960

Fuente: Mercado Pago – Link de pago

Cuánto deja cada centro

Costos fijos de la plataforma (se reparten entre todos los centros): Supabase Pro + Vercel Pro ≈$45 USD (≈$180.000 COP) + Resend Pro ≈$80.000 + dominio ≈$5.000 al mes ≈ **$265.000 COP al mes**.

Por cada centro en Plan Básico mensual: $280.000 − comisión (≈$11.000) − WhatsApp (≈$22.000) ≈ **$247.000 COP al mes** para cubrir los costos fijos.

Punto de equilibrio: con **2 centros en Plan Básico** ya se cubren los costos fijos; desde el tercero, casi todo es ganancia. En Plan Pro, un solo centro (≈$440.000 netos) los cubre.

Comparación

- Sin tocar nada: ≈$45/mes.
- VPS propio con Postgres: ≈$6–15/mes (estimado).
- Diferencia: ≈$30–40/mes. Si la migración toma varios días de trabajo, el ahorro la paga en cerca de un año.

4. Recomendación

Si el objetivo es **ahorrar costos y tener más control**, la opción recomendada es **reescribir la capa de datos para Postgres puro** y desplegar todo en un VPS propio. No se recomienda Supabase autoalojado: son más de diez servicios que hay que actualizar, respaldar y vigilar, y requiere un servidor más grande, lo que quita el ahorro y el control que se buscan.

Si el tiempo es lo más importante, quedarse en Supabase Pro + Vercel Pro (≈$45/mes) es válido y no requiere cambios.

Plan de la migración (en una rama aparte, sin tocar `main` ni Supabase hasta aprobar)

1. **Datos:** cambiar las ≈56 consultas de `supabase-js` por `pg` o Drizzle. Cada petición abre una transacción con el usuario declarado para que RLS siga protegiendo a cada centro. Las rutas del servidor que hoy usan la service role usan una conexión propia.
2. **Login:** reemplazar el de Supabase por uno propio con sesiones en cookie y contraseñas con Argon2 o bcrypt, y crear y borrar usuarios desde la app. Es la parte más delicada y la que requiere más pruebas.
3. **Logos:** guardarlos en disco, en una carpeta que sirve el mismo servidor.
4. **Tiempo real:** reemplazarlo por consultas periódicas cada pocos segundos.
5. **Despliegue:** Docker con Next.js y Postgres, un proxy con HTTPS (Caddy o Nginx) y respaldos automáticos de la base.

Qué pasa a ser responsabilidad tuya

- Respaldos de la base (y probar que se pueden restaurar).
- Actualizaciones de seguridad del servidor y de Postgres.
- Monitoreo: saber si el servidor o los recordatorios dejaron de funcionar.
- Protección de los datos de pacientes (Ley 1581 de 2012 en Colombia).

---

5. Pendientes de seguridad independientes de la migración

1. **Rotar las credenciales de Twilio.** El `TWILIO_AUTH_TOKEN` estuvo en texto plano en `.env.local` y apareció en la salida de búsquedas durante el trabajo con el asistente. Genera uno nuevo en la consola de Twilio.
2. Revisar `npm audit` y aplicar solo arreglos sin cambios mayores.
3. Mantener `.env.local` fuera de Git (ya está en `.gitignore`).
4. Fijar la URL de ngrok si se sigue probando en local (`ngrok http 3001 --url=<tu-dominio>`).
5. Revisar con asesoría legal la política de privacidad antes de operar con pacientes reales.

6. Datos útiles del entorno de pruebas

- **Webhook de Twilio (local):** `https://<tu-dominio-ngrok>/api/whatsapp/webhook`, método POST.
- **Variables en `.env.local` para ngrok:** `APP_URL` y `TWILIO_WEBHOOK_URL` (con la misma URL de ngrok).
- **Número de WhatsApp por centro:** si un centro tiene `whatsapp_from` guardado, ese número tiene prioridad sobre `TWILIO_WHATSAPP_FROM`. Si el número no está registrado en Twilio, el envío falla con "Twilio could not find a Channel with the specified From address". Desde el 15 de octubre de 2026 los dos centros tienen el campo vacío y usan el número de la plataforma (sandbox).
- **Producción:** el sandbox de Twilio solo envía a números que se unieron con `join …` (caduca a las 72 horas) y que escribieron en las últimas 24 horas. Para pacientes reales hacen falta un número propio aprobado y plantillas aprobadas (`TWILIO_WHATSAPP_CONTENT_SID`, `TWILIO_CAMPANA_CONTENT_SID`, `TWILIO_CARNE_CONTENT_SID`). Hoy hay una plantilla por tipo para toda la plataforma: si cada centro tiene su propia cuenta de Meta, la app tendría que guardar la plantilla de cada centro.
- **Estado de entrega:** solo funciona en producción (https). Twilio llama a `https://<dominio>/api/whatsapp/estado` con la firma `X-Twilio-Signature`; en local con ngrok, `APP_URL` debe ser la URL https de ngrok.
- **Mercado Pago:** los links de pago están en `LINKS_PAGO` (`src/lib/suscripcion.ts`). Para el modo automático: crear una aplicación Checkout Pro, poner `MERCADOPAGO_ACCESS_TOKEN` y `MERCADOPAGO_WEBHOOK_SECRET`, y configurar el webhook `https://<dominio>/api/suscripcion/webhook` con el evento "Pagos".